GitHub Copilot Capstone Project

Agentic SDLC Use Case: Automated Documentation Sync

Team Exercise | GitHub Copilot Agent Mode

Project Overview

In this capstone, let us build an Agentic SDLC Pipeline implementation from scratch using GitHub Copilot's capabilities and features. SDLC implementation should entirely drive through GitHub Copilot Agens, Prompts, Instructions, Skills and Hooks as per the need.

You will practice the complete software delivery lifecycle — from writing requirements to merging a production-ready PR — with GitHub Copilot as your AI pair programmer at every step.

Step 1: Requirements

Use GitHub Copilot Chat/CLI to collaboratively define and document the functional and non-functional requirements for the given User Story.

- Read a new User Story from JIRA/Confluence/word document.
- Let Copilot ask or clarify questions on the same and let user respond to those questions.
- Capture the final requirements in requirements.md and commit the file.

Step 2: Architecture

Use GitHub Copilot Chat/CLI to design the high-level system architecture. Ask Copilot to propose component diagrams, technology choices, and data flow.

- Ask for an architecture recommendation based on your requirements.md.
- Document the proposed architecture in architecture.md.
- Identify the key components and their responsibilities.

Step 3: Design Review

Use GitHub Copilot Chat/CLI to conduct a structured design review of the architecture before writing any production code. Treat Copilot as a senior reviewer.

- Share architecture.md with Copilot Chat and ask it to identify risks and gaps.
- Document review findings and agreed design decisions in design-review.md.
- Update architecture.md if any issues are found.

Step 4: Implementation Planning

Use GitHub Copilot Chat/CLI to break the approved architecture down into a prioritised, dependency-ordered task list.

- Ask Copilot to generate a task breakdown from architecture.md.
- Document the plan in impl-plan.md, ordered by dependency.
- Identify any blocked tasks that cannot start until another finishes.

Step 5: Implementation

Use GitHub Copilot Chat/CLI to implement the changes suggested by GHCP and approved by the human in the loop.

GitHub Copilot features to use in this step:

Step 6: Review

Use GitHub Copilot Chat/CLI to perform a structured code review of your own implementation before creating the PR. Copilot acts as a peer reviewer.

Code review checklist — ask Copilot to evaluate each area:

Review Area	Review Question
Correctness	Does each component behave as specified in requirements.md?
Security	Are secrets excluded from output? Is user input validated?
Error Handling	Are all API failures, missing files, and empty repos handled gracefully?
Test Coverage	Do tests cover the happy path AND the 'Not Found' / missing-field edge cases?
Code Clarity	Are function names self-explanatory? Is logic easy to follow without comments?
DRY Principle	Is there duplicated logic that Copilot can refactor into a shared function?
Dependency Safety	Does Copilot flag any known-vulnerable package versions?

Step 7: Verify

Use GitHub Copilot to generate and run a comprehensive verification suite. Verify both the code (unit + integration tests) and the final output document (content quality check).

Step 8: PR Using Agentic SDLC

Use GitHub Copilot Agent Mode to create the Pull Request — including the PR description, changelog entry, and review checklist — completing the full agentic SDLC cycle.

Required PR description sections (Copilot must generate all of these):

- Summary — 2-3 sentence overview of what was built and why.
- Changes Made — bulleted list of all files added/modified and the reason.
- Test Evidence — paste the test run output or link to CI results.
- Known Limitations — anything marked 'Not Found' or out of scope.
- Reviewer Checklist — a tick-list the reviewer must complete before approving.ss